

University OF Engineering and Technology, Lahore
Computer Engineering Department

Course Name: PF	Course Code: CMPE-112L
Assignment Type: lab	Dated: 30 MARCH2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 5	CLOs to be covered: CLO2
Lab Title: functions	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 5

Lab Goals/Objectives:

- **Reusability (DRY - Don't Repeat Yourself):** Write the logic once, use it an infinite number of times across your program.
- **Decomposition (Modularity):** Breaking a massive, complex project down into smaller, manageable, and isolated chunks (e.g., having separate functions for `calculate_score ()`, `validate_user ()`, and `save_to_database ()`).
- **Readability & Maintainability:** It is much easier to read code that calls well-named functions than to scroll through a giant wall of unstructured code. If a bug pops up, you only have to fix it in one specific function.
- **Abstraction:** You only need to know *what* the function returns and *what inputs* it requires, without constantly worrying about *how* it does it under the hood

Equipment Required:

- Computer/Laptop
- Python 3.x
- VS Code / PyCharm / IDLE

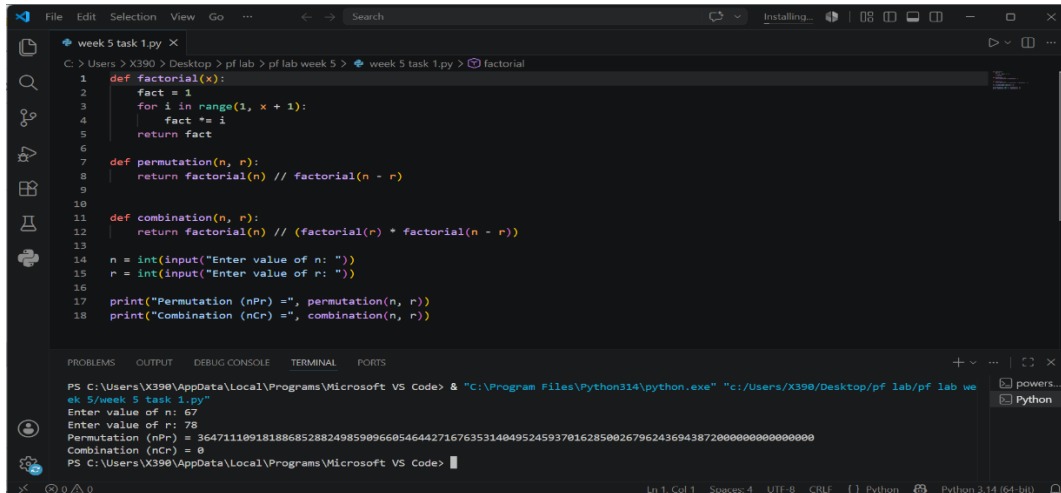
THEORY:

The core theory of functions revolves around establishing a predictable relationship that maps inputs to unique outputs, acting as a reliable rule or machine in both mathematics and computer science. Mathematically, a function connects every element of a starting domain to exactly one target value in the codomain, allowing us to model how changing variables impact one another with absolute precision. In programming, this exact concept translates into software modularity, where complex systems are decomposed into isolated, single-purpose blocks of code. By packaging specific logic into these distinct modules, developers achieve code reusability—eliminating repetitive tasks by writing a process once and running it anywhere with varying arguments. Ultimately, whether analyzing abstract data structures or building real-world software applications, the overriding objective of a function is to systematically transform inputs into predictable, structured data based on a strict set of rules.

TASK NO 1:

Write a program that calculates the number of permutations (nPr) and combinations (nCr) based on user-defined inputs. The program must prompt the user to enter the total number of items (n) and the number of items to select (r)

RESULT:



```
1 def factorial(x):
2     fact = 1
3     for i in range(1, x + 1):
4         fact *= i
5     return fact
6
7 def permutation(n, r):
8     return factorial(n) // factorial(n - r)
9
10
11 def combination(n, r):
12     return factorial(n) // (factorial(r) * factorial(n - r))
13
14 n = int(input("Enter value of n: "))
15 r = int(input("Enter value of r: "))
16
17 print("Permutation (nPr) =", permutation(n, r))
18 print("Combination (nCr) =", combination(n, r))
```

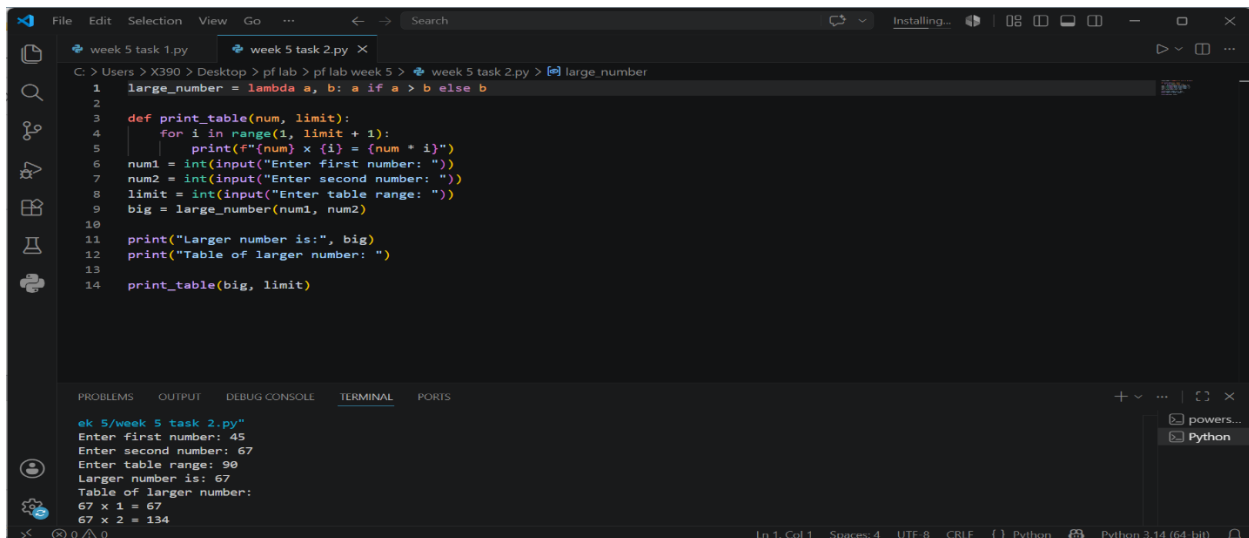
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "C:\Users\X390\Desktop\pf lab\pf lab week 5\week 5 task 1.py"

Enter value of n: 67
Enter value of r: 78
Permutation (nPr) = 364711109181868528824985909660546442716763531404952459378162850026796243694387200000000000000
Combination (nCr) = 0

TASK NO 2:

Write a program that calculates the number of permutations (nPr) and combinations (nCr) based on user-defined inputs. The program must prompt the user to enter the total number of items (n) and the number of items to select (r)

RESULT:



```
1 large_number = lambda a, b: a if a > b else b
2
3 def print_table(num, limit):
4     for i in range(1, limit + 1):
5         print(f"{num} x {i} = {num * i}")
6
7 num1 = int(input("Enter first number: "))
8 num2 = int(input("Enter second number: "))
9 limit = int(input("Enter table range: "))
10 big = large_number(num1, num2)
11
12 print("Larger number is:", big)
13 print("Table of larger number: ")
14 print_table(big, limit)
```

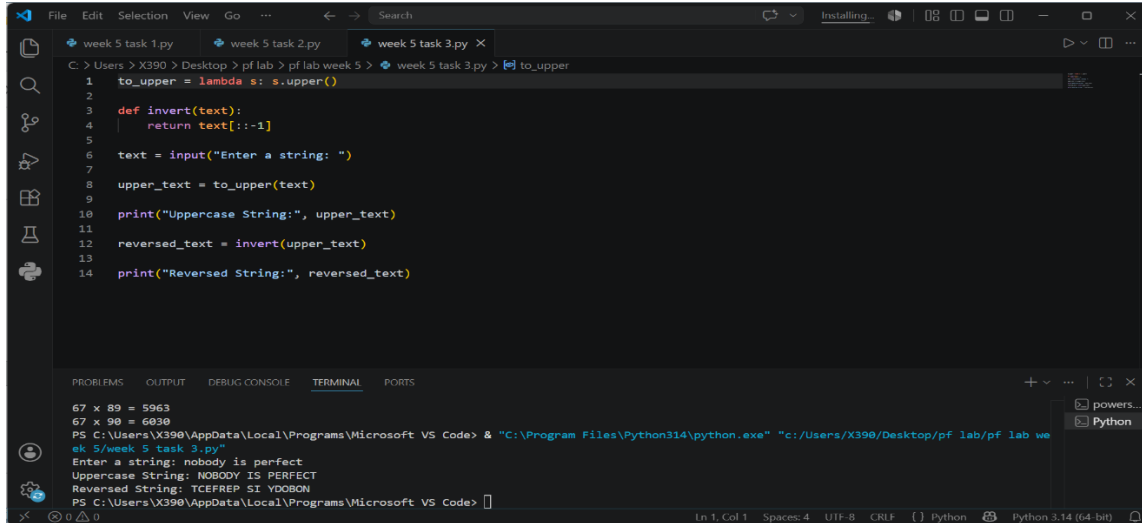
ek 5/week 5 task 2.py

Enter first number: 45
Enter second number: 67
Enter table range: 90
Larger number is: 67
Table of larger number:
67 x 1 = 67
67 x 2 = 134

TASK NO 3:

Write a program that takes a string input from the user and performs sequential text manipulations using lambda functions and slicing methods

RESULT:



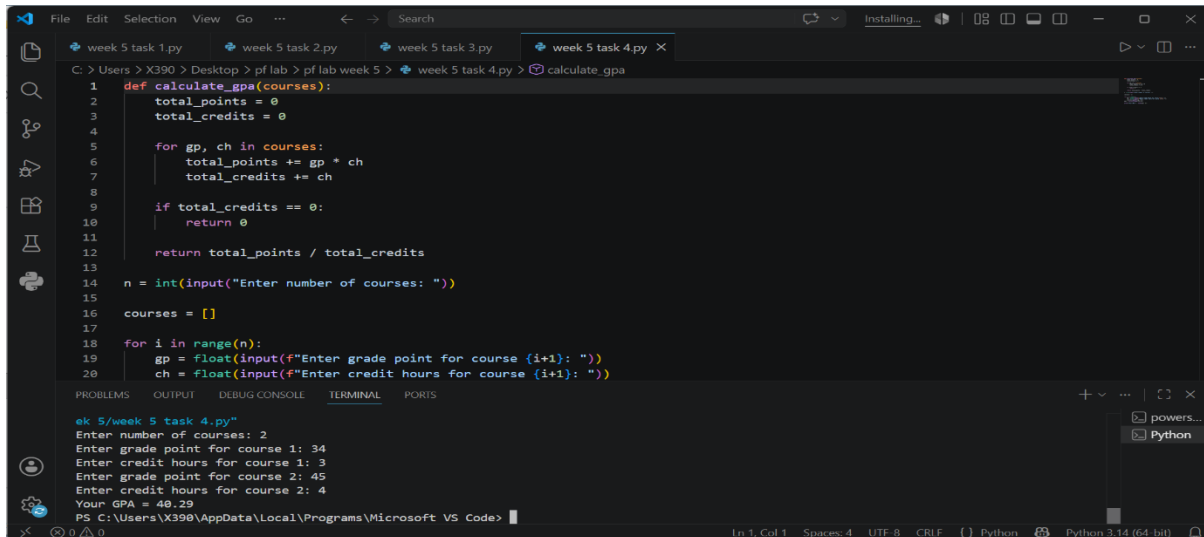
```
File Edit Selection View Go ... Search
week 5 task 1.py week 5 task 2.py week 5 task 3.py X
C:\Users\X390\Desktop> pf lab > pf lab week 5 > week 5 task 3.py > to_upper
1 to_upper = lambda s: s.upper()
2
3 def invert(text):
4     return text[::-1]
5
6 text = input("Enter a string: ")
7
8 upper_text = to_upper(text)
9
10 print("Uppercase String:", upper_text)
11
12 reversed_text = invert(upper_text)
13
14 print("Reversed String:", reversed_text)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
67 x 89 = 5963
67 x 90 = 6030
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab we
ek 5/week 5 task 3.py"
Enter a string: nobody is perfect
Uppercase String: NOBODY IS PERFECT
Reversed String: TCEPREP SI YDOBON
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit)
```

TASK NO 4:

Write a program that calculates a student's semester Grade Point Average (GPA) by tracking courses dynamically via data collections

RESULT:



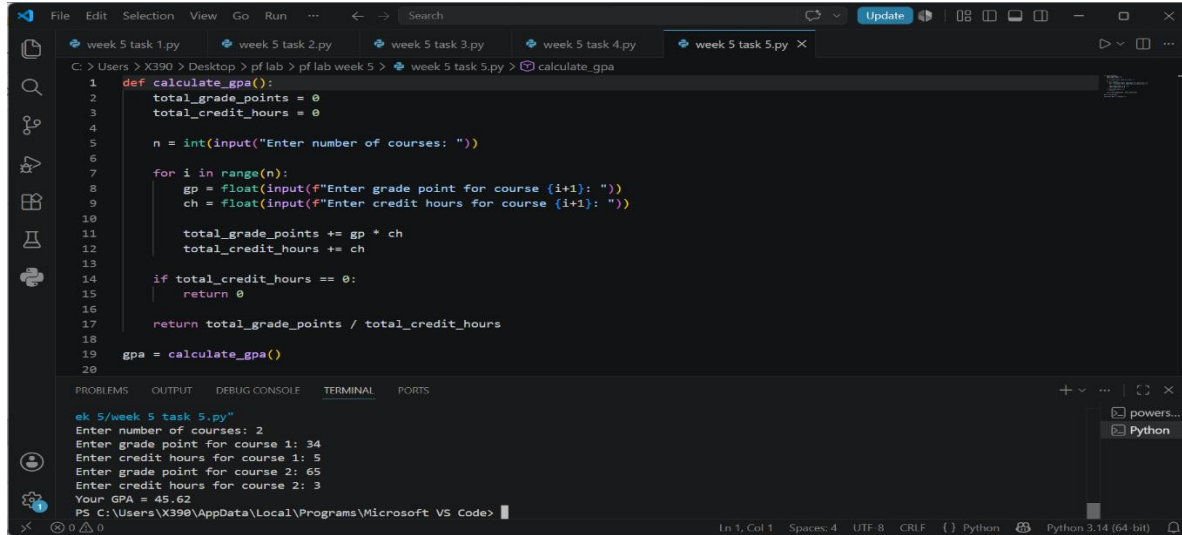
```
File Edit Selection View Go ... Search
week 5 task 1.py week 5 task 2.py week 5 task 3.py week 5 task 4.py X
C:\Users\X390\Desktop> pf lab > pf lab week 5 > week 5 task 4.py > calculate_gpa
1 def calculate_gpa(courses):
2     total_points = 0
3     total_credits = 0
4
5     for gp, ch in courses:
6         total_points += gp * ch
7         total_credits += ch
8
9     if total_credits == 0:
10        return 0
11
12    return total_points / total_credits
13
14 n = int(input("Enter number of courses: "))
15
16 courses = []
17
18 for i in range(n):
19     gp = float(input(f"Enter grade point for course {i+1}: "))
20     ch = float(input(f"Enter credit hours for course {i+1}: "))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ek 5/week 5 task 4.py"
Enter number of courses: 2
Enter grade point for course 1: 34
Enter credit hours for course 1: 3
Enter grade point for course 2: 45
Enter credit hours for course 2: 4
Your GPA = 40.29
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit)
```

TASK NO 5:

Write a program to calculate a student's semester GPA without storing the intermediate data inside a list structure, optimizing memory consumption

RESULT:



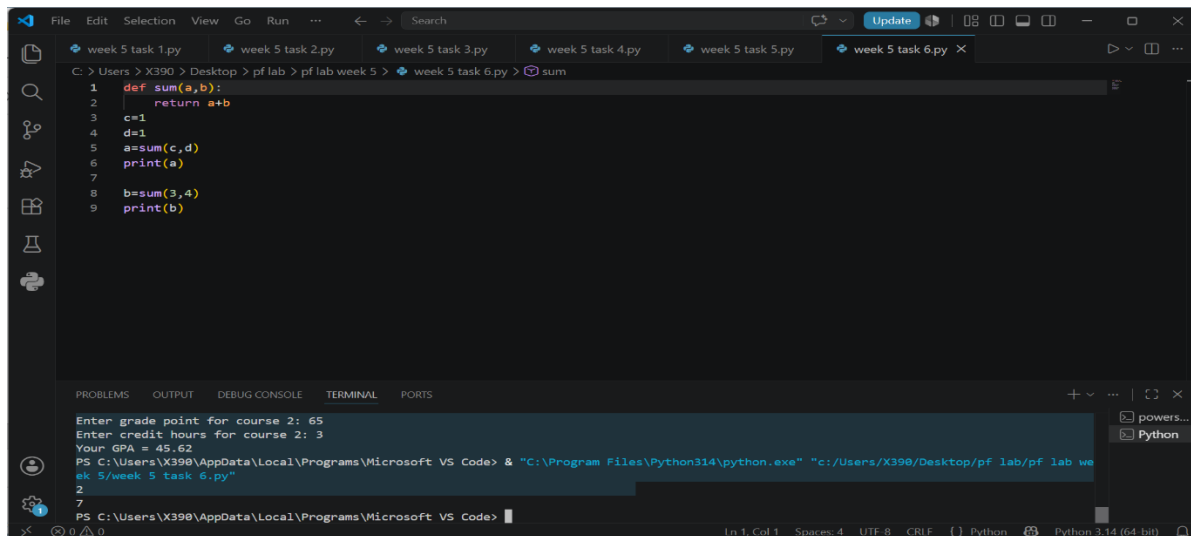
The screenshot shows a Visual Studio Code editor with a Python file named 'calculate_gpa.py'. The code defines a function 'calculate_gpa()' that takes the number of courses as input and iterates through them, calculating the total grade points and total credit hours. The GPA is then calculated as the total grade points divided by the total credit hours. The terminal output shows the program running successfully with the following input and output:

```
ek 5/week 5 task 5.py
Enter number of courses: 2
Enter grade point for course 1: 34
Enter credit hours for course 1: 5
Enter grade point for course 2: 65
Enter credit hours for course 2: 3
Your GPA = 45.62
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 6:

Write a program that demonstrates the fundamental concepts of modular programming, specifically focusing on how functions accept parameters and return values

RESULT:



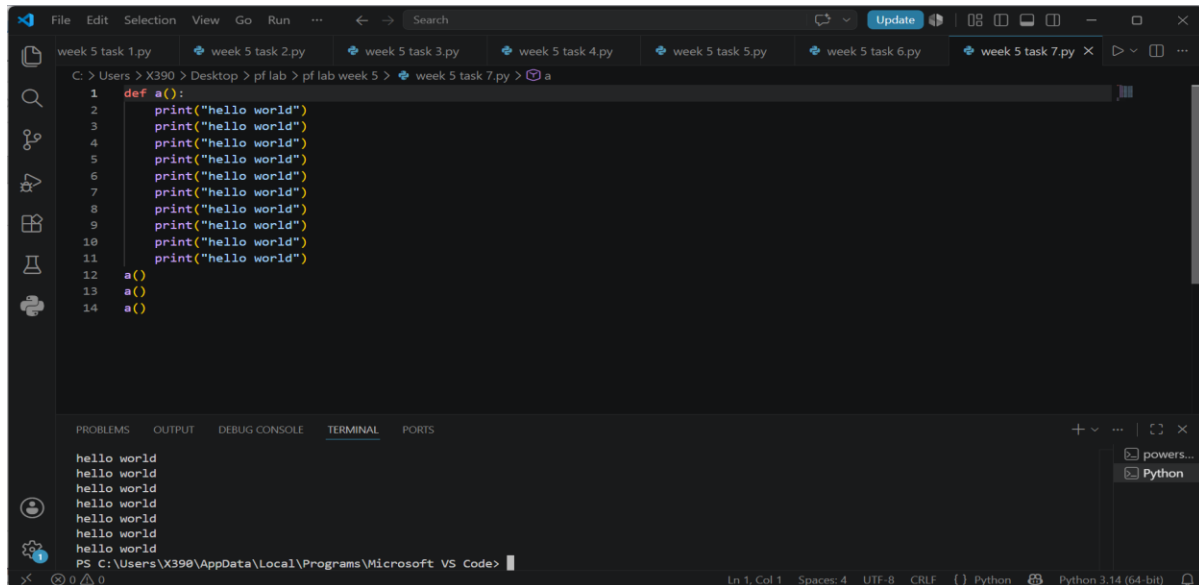
The screenshot shows a Visual Studio Code editor with a Python file named 'sum.py'. The code defines a function 'sum(a,b)' that returns the sum of two numbers. It then demonstrates the function's usage by calling it with different parameters and printing the results. The terminal output shows the program running successfully with the following input and output:

```
Enter grade point for course 2: 65
Enter credit hours for course 2: 3
Your GPA = 45.62
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf_lab/pf_lab we
ek 5/week 5 task 6.py"
2
7
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 7:

Write a program that demonstrates code reusability and execution flow control by packaging a series of sequential output operations inside a named code block

RESULT:



The screenshot shows the Microsoft Visual Studio Code editor interface. The top panel displays several open files, with 'week 5 task 7.py' selected. The editor window shows the following Python code:

```
1 def a():
2     print("hello world")
3     print("hello world")
4     print("hello world")
5     print("hello world")
6     print("hello world")
7     print("hello world")
8     print("hello world")
9     print("hello world")
10    print("hello world")
11    print("hello world")
12
13 a()
14 a()
15 a()
```

The bottom panel shows the 'TERMINAL' tab, which displays the output of the program:

```
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
```

The status bar at the bottom indicates the file is 'Python 3.14 (64-bit)' and the current cursor position is 'Ln 1, Col 1'.

CONCLUSION:

- Functions serve as the fundamental building blocks for organizing complexity in both mathematics and software design. They bridge abstract mathematical relationships with practical application by transforming inputs into predictable, structured outputs. By enforcing strict rules of uniqueness and modularity, they ensure that data processing remains reliable and error-free. In development, they turn chaotic walls of text into clean, reusable, and easily maintainable functional modules. Ultimately the use of functions is the definitive method for breaking down massive, complex systems into manageable pieces.